

Algorithms Mock Exam SS 2026 — Complete Study Guide

Reverse-engineered from Exam ID 024-A (90 points, 9 tasks, 20 pages)

1. EXAM PAPER ANALYSIS

#	Task	Topic	Prerequisites	Difficulty	Marks	Type
1a	D&C phases	Divide & Conquer	None	Easy	3	Recall/theory
1b	MergeSort recurrence	Recurrences	Task 1a	Easy	1	Formula
1c	QuickSort pseudocode	D&C, arrays	Task 1a, given Partition	Medium	4	Implementation
2a	Recursion tree ($T[n]=5T[n/3]+c \log n$)	Recursion trees	Task 1	Medium	5	Calculation
2b	Non-recursive formula from tree	Summations	Task 2a	Medium	2	Derivation
3a	Edit Distance definition	DP problem catalogue	None	Easy	3	Recall
3b	Matrix Chain Multiplication definition	DP problem catalogue	None	Easy	2	Recall
3c	TSP definition	Graph problems	None	Easy	2	Recall
3d	Independent Set Decision definition	Graph problems, NP	None	Easy	3	Recall
4a	Formalize MAXIS(v)	Tree DP	Task 3d	Medium	1	Conceptual

#	Task	Topic	Prerequisites	Difficulty	Marks	Type
4b	Use MAXIS values to get answer	Tree DP	Task 4a	Easy	1	Conceptual
4c	Recursive formula for MAXIS(v)	Tree DP	Task 4a/b	Hard	3	Derivation
4d	Execute TreeMaxIS on a tree	Tree DP	Task 4c	Hard	6	Calculation
5a	3 phases of FFT polynomial mult.	FFT	Task 1a pattern	Medium	6	Recall/theory
5b	Execute FFT mult. on two linear polys	FFT, complex roots	Task 5a	Medium	4	Calculation
5c	Prove product of n-th roots of unity	Complex numbers, proof	Task 5b	Hard	7	Proof
6a	Run Huffman algorithm (build table)	Greedy, priority queues	None	Medium	4	Calculation
6b	Draw Huffman coding tree	Task 6a	Medium	4	Construction	
6c	Compute Huffman code cost	Task 6a/b	Easy	2	Calculation	
7a	Spanning tree / cost definitions	Graph theory	None	Easy	2	Recall
7b	State the Cut Property	MST theory	Task 7a	Hard	4	Theorem recall
7c(i-ii)	Execute Kruskal's +	Kruskal's, Union-Find	Task 7b	Medium	4	Calculation

#	Task	Topic	Prerequisites	Difficulty	Marks	Type
Union-Find						
8a	Dijkstra runtime w/ binary heap	Priority queues, complexity	None	Easy	1	Recall
8b	Execute Dijkstra's algorithm	Shortest paths	Task 8a	Medium	5	Calculation
8c	Interpret $\text{dist}[t]=\infty$	Graph reachability	Task 8b	Medium	2	Conceptual
9a	Meaning of $P[S,w]$ in TSP-DP	DP over subsets	Task 3c	Hard	3	Conceptual
9b	Prove Vertex Cover, MST-Decision $\in$ NP	NP, certificates	Complexity theory	Hard	6	Proof

Total: 90 points across 9 tasks.

Patterns detected across the paper

- Every algorithm topic is tested at 3 levels:** (1) state/define the concept, (2) reproduce the algorithm/pseudocode/recurrence, (3) hand-execute it on a concrete instance. Expect this 3-layer pattern to repeat on the real exam.
- "Fill in the blank" recall questions** (Task 3, phases of D&C, Cut Property) are the cheapest points on the exam — pure memorization, no reasoning required.
- DP problems always tested via "Input/Output" templates** — you are expected to know the *exact formal specification* (not just the idea) of Edit Distance, Matrix Chain, TSP, Independent Set, Vertex Cover, MST-Decision.
- Every graph algorithm (Kruskal, Dijkstra, Huffman, Tree-DP) is tested by hand-execution on a small instance** — this is the single highest-value skill to drill.
- Proofs appear twice:** an algebraic proof (roots of unity) and a complexity-theory proof (NP membership). Both follow a fixed template you can memorize.
- Recurrences/recursion trees are tested abstractly** (symbolic $\boxed{c}$, $\boxed{n=3^x}$) rather than with concrete numbers — you must be comfortable working with symbols, not just plugging in numbers.

2. RECONSTRUCTED SYLLABUS

Unit A — Algorithm Design Paradigms

- Divide & Conquer (phases, MergeSort, QuickSort, Partition)
- Recurrences & Recursion Trees (deriving runtime bounds)
- Greedy Algorithms (Huffman coding, Kruskal's MST)
- Dynamic Programming (Edit Distance, Matrix Chain Multiplication, Tree Independent Set, TSP/Held-Karp)

Unit B — Graph Algorithms

- Minimum Spanning Trees (Kruskal's Algorithm, Cut Property, Union-Find)
- Single-Source Shortest Paths (Dijkstra's Algorithm)

Unit C — Advanced Algorithmic Techniques

- Fast Fourier Transform & Polynomial Multiplication
- Complex n -th roots of unity

Unit D — Complexity Theory

- NP, certificates, polynomial-time verification
- Classic NP problems: Vertex Cover, Independent Set, TSP, MST-Decision

3. TOPIC PRIORITY RANKING (by marks, highest first)

Rank	Topic	Marks	% of exam
1	FFT & Polynomial Multiplication	17	19%
2	Tree DP (Maximum Independent Set)	11	12%
3	Problem Definitions (Edit Dist., MCM, TSP, IS)	10	11%
4	Huffman Algorithm	10	11%
5	MST / Kruskal's + Cut Property	10	11%
6	TSP-DP + NP-completeness proofs	9	10%

Rank	Topic	Marks	% of exam
7	Divide & Conquer basics + QuickSort	8	9%
8	Dijkstra's Algorithm	8	9%
9	Recursion Trees	7	8%

If your time is short, study in this order: FFT → Tree DP → Huffman → MST/Kruskal → Problem Definitions → NP proofs/TSP-DP → Dijkstra → D&C/QuickSort → Recursion Trees.

Note: even though FFT ranks #1 by raw marks, it is conceptually self-contained (3 sub-parts). Tree DP and NP proofs are higher-risk because they require genuine derivation under time pressure, not just recall — budget extra practice time there.

4. DETAILED STUDY NOTES

4.1 Divide & Conquer

Beginner explanation: Break a big problem into smaller versions of the *same* problem, solve those smaller versions (possibly by breaking them down further), then stitch the small answers back into a big answer.

The three phases (MEMORIZE):

1. **Divide** — split the problem instance into smaller sub-instances (usually of the same type).
2. **Conquer** — solve the sub-instances recursively; if a sub-instance is small enough, solve it directly (base case).
3. **Combine** — merge/combine the solutions of the sub-instances into a solution for the original instance.

MergeSort recurrence (Task 1b logic): MergeSort splits $A[1..n]$ into $A[1..[n/2]]$ and $A[[n/2]+1..n]$, recursively sorts both, then merges in $O(n)$ time.

$$T[n] \leq T\left[\left\lfloor \frac{n}{2} \right\rfloor\right] + T\left[\left\lceil \frac{n}{2} \right\rceil\right] + O(n)$$

(Often simplified/written as $T[n] \leq 2T[n/2] + cn$.)

QuickSort — how examiners test it: They give you $\text{PARTITION}(a, b, p)$ (rearranges $A[a..b]$ around pivot $A[p]$, returns final pivot index ℓ) and ask you to write QUICKSORT from scratch. This is a **pattern you must memorize cold**:

```

QUICKSORT(a, b):
1  if a < b:
2      p ← b                // choose pivot (commonly the last element)
3      ℓ ← PARTITION(a, b, p)
4      QUICKSORT(a, ℓ - 1)
5      QUICKSORT(ℓ + 1, b)

```

Key exam insight: after PARTITION , $A[\ell]$ is in its final sorted position, so you recurse on the two disjoint ranges *excluding* ℓ .

4.2 Recursion Trees

Core intuition: A recursion tree is a picture of the recurrence: the root is the original call, its children are the recursive sub-calls, and so on until you hit the base case. Summing the "cost per level" over all levels gives you the total runtime.

General method for $T[n] = a \cdot T[n/b] + f(n)$:

1. **Level k has** a^k **nodes**, each representing a subproblem of size n/b^k .
2. **Depth** d : the recursion bottoms out when $n/b^d = 1$, i.e. $d = \log_b(n)$.
3. **Number of leaves** = $a^d = a^{\log_b n} = n^{\log_b a}$.
4. **Cost per level** = (number of nodes at that level) $\times$ (merge cost per call at that level).
5. **Total cost** = sum of costs over all levels (levels 0 to $d-1$) **plus** the total leaf cost.

Worked pattern — Task 2 style: $T[1]=c$, $T[n] = 5 \cdot T[n/3] + c \cdot \log n$, for $n = 3^x$.

Level	# subproblems	Size of each	Merge cost <i>per call</i>	Total merge cost at level
0	1	n	$c \cdot \log n$	$c \cdot \log n$
1	5	$n/3$	$c \cdot \log(n/3)$	$5 \cdot c \cdot \log(n/3)$
2	$25 = 5^2$	$n/9$	$c \cdot \log(n/9)$	$25 \cdot c \cdot \log(n/9)$
k	5^k	$n/3^k$	$c \cdot \log(n/3^k)$	$5^k \cdot c \cdot \log(n/3^k)$
d (leaves)	5^d	1	c (base case)	$5^d \cdot c$

- **Depth:** $n/3^d = 1 \Rightarrow d = \log_3 n$.
- **#leaves:** $5^d = 5^{\log_3 n} = n^{\log_3 5}$ (since $\log_3 5 \approx 1.465$).

Non-recursive (unsimplified) formula:

$$T[n] = \sum_{k=0}^{\log_3 n - 1} 5^k \cdot c \log\left(\frac{n}{3^k}\right) + n^{\log_3 5} \cdot c$$

How to build this fast under time pressure: write the general row $\boxed{\text{level } k \rightarrow a^k \text{ calls} \rightarrow \text{size } n/b^k \rightarrow \text{merge cost } c \cdot f(n/b^k)}$, fill in a, b, f from your specific recurrence, then set $\boxed{n/b^d = 1}$ to get $\boxed{d}$.

4.3 Standard Problem Definitions (memorize verbatim style)

Examiners literally ask "Input: ___ Output: ___" — treat these like vocabulary flashcards.

Edit Distance

- **Input:** Two strings $\boxed{A, B}$ (over an alphabet Σ).
- **Output:** The edit distance between $\boxed{A}$ and $\boxed{B}$.
- **Definition:** The edit distance of two strings $\boxed{A, B}$ is the **minimum number of single-character insertions, deletions, and substitutions** needed to transform $\boxed{A}$ into $\boxed{B}$.

Matrix Chain Multiplication

- **Input:** A sequence of matrices $\boxed{A_1, A_2, \dots, A_n}$ (equivalently, dimensions $\boxed{p_0, p_1, \dots, p_n}$ where $\boxed{A_i}$ has size $\boxed{p_{i-1} \times p_i}$).
- **Output:** A parenthesization of the product $\boxed{A_1 \cdot A_2 \cdot \dots \cdot A_n}$ that **minimizes the total number of scalar multiplications**.

Traveling Salesman Problem (TSP)

- **Input:** A graph $\boxed{G=(V, E)}$ with a cost function $\boxed{c: E \rightarrow \mathbb{R}^+_{\circ}}$ (typically complete graph = all pairwise costs given).
- **Output:** A **Hamiltonian cycle** (visits every vertex exactly once, returns to start) of **minimum total cost**.

Independent Set Decision Problem

- **Input:** An undirected graph $\boxed{G=(V, E)}$, an integer $\boxed{k}$.
- **Output:** Is there an independent set $\boxed{S \subseteq V}$ with $\boxed{|S| \geq k}$?
- **Definition:** An independent set in an undirected graph $\boxed{G=(V, E)}$ is a subset $\boxed{S \subseteq V}$ such that **no two vertices in $\boxed{S}$ are connected by an edge of $\boxed{E}$** .

- **Input:** An undirected graph $G=(V, E)$, integer k .
- **Output:** Is there a set $C \subseteq V$ of size $|C| \leq k$ such that every edge in E is incident to some $v \in C$?

Minimal Spanning Tree Decision Problem

- **Input:** A graph $G=(V, E, c)$ with non-negative integer edge weights, integer k .
 - **Output:** Is there a spanning tree of G with weight at most k ?
-

4.4 Dynamic Programming on Trees (Maximum Independent Set)

Setup: T is a tree rooted at r . For vertex v , T_v = subtree rooted at v ; $N(v)$ = children of v in the rooted tree.

Core intuition: For every vertex, you have a binary choice — *include it in the independent set, or don't*.

- If you **include** v , you cannot include any child of v (they'd be adjacent to v), but you *can* include grandchildren.
- If you **exclude** v , each child subtree is free to take its own best (independently) solution.

Formalization (Task 4a/b):

| $MAXIS(v)$:= the size of the **maximum independent set in the subtree** T_v rooted at v .

Once $MAXIS(v)$ is known for every vertex, the answer for the whole tree is simply:

| $MAXIS(r)$ — the value computed at the root.

Recursive formula (Task 4c) — MEMORIZE:

$$MAXIS(v) = \max \left\{ 1 + \sum_{w \in N(v)} \sum_{x \in N(w)} MAXIS(x), \sum_{w \in N(v)} MAXIS(w) \right\}$$

- First term: **include** $v \rightarrow +1$, skip children $N(v)$, add the best solutions of *grandchildren* $N(w)$ for each child w .
- Second term: **exclude** $v \rightarrow$ add up the best independent-set value of each child subtree directly.
- **Base case:** if v is a leaf, $MAXIS(v) = 1$.

How to execute **TreeMaxIS** by hand (bottom-up, post-order):

1. Start at the **leaves**: $\text{MAXIS}(\text{leaf}) = 1$.
2. Move up level by level. For each internal node v , once *all* its children's (and grandchildren's) values are known, apply the formula above.
3. Continue until you reach the root; the value at the root is your final answer.

Worked illustrative example (small binary-ish tree, root r with children a, b ; a has children d, e ; b is a leaf):



- $\text{MAXIS}(d) = 1, \text{MAXIS}(e) = 1, \text{MAXIS}(b) = 1$ (leaves)
- $\text{MAXIS}(a) = \max(1 + [\text{grandchildren of } a \text{ via } d, e \rightarrow \text{none}] , \text{MAXIS}(d) + \text{MAXIS}(e)) = \max(1, 1+1) = 2$
- $\text{MAXIS}(r) = \max(1 + \text{MAXIS}(d) + \text{MAXIS}(e) [\text{grandchildren via } a] + [\text{grandchildren via } b \rightarrow \text{none}] , \text{MAXIS}(a) + \text{MAXIS}(b)) = \max(1 + 1 + 1 , 2 + 1) = \max(3, 3) = 3$

Exam tip: Always write the value **inside the dotted circle at every vertex**, including leaves (worth easy partial credit), and show the two candidate values (include/exclude) as scratch work next to each internal node — many rubrics give partial credit for showing the $\max()$ comparison even if the final number is wrong.

Complexity: $O(n)$ — every vertex and edge is visited a constant number of times in the post-order traversal.

4.5 FFT & Polynomial Multiplication (highest-value topic — 17 pts)

Core intuition: Multiplying two degree- $(n-1)$ polynomials the naive way (coefficient by coefficient) costs $O(n^2)$. FFT gets this down to $O(n \log n)$ using a clever trick: represent polynomials by their **values at special points** (roots of unity) instead of by coefficients, because *point-value multiplication is just pointwise multiplication* ($O(n)$), and converting between representations can be done fast via FFT.

The three phases (MEMORIZE — Task 5a):

1. **Evaluation (FFT):** Evaluate both $A(x)$ and $B(x)$ at the n -th roots of unity $(\omega^0, \omega^1, \dots, \omega^{n-1})$ using the Fast Fourier Transform, in $O(n \log n)$ time. (This gives the *point-value representation* of A and B .)
2. **Pointwise Multiplication:** Multiply the evaluations pairwise: $C(\omega^k) = A(\omega^k) \cdot B(\omega^k)$ for each $k = 0, \dots, n-1$, in $O(n)$ time.
3. **Interpolation (Inverse FFT):** Recover the **coefficient representation** of $C(x)$ from its n point-values using the inverse FFT, in $O(n \log n)$ time.

Total runtime: $O(n \log n)$.

How to determine n (Task 5b pattern):

- If $\deg(A) = d_A$ and $\deg(B) = d_B$, then $\deg(C) = d_A + d_B$, so C needs $d_A + d_B + 1$ coefficients to be uniquely represented.
- n must be a **power of 2** (for the FFT to work) and $n \geq d_A + d_B + 1$.
- **Rule:** round $d_A + d_B + 1$ up to the next power of 2.

Example: $A(x) = 4x+3$ (degree 1), $B(x) = 7x+9$ (degree 1) $\rightarrow \deg(C) = 2 \rightarrow$ need 3 points minimum $\rightarrow$ smallest power of 2 that is ≥ 3 is $n = 4$.

Marking the n -th roots of unity on the unit circle: the n -th roots of unity are $\omega^k = e^{2\pi i k/n}$ for $k = 0, \dots, n-1$, evenly spaced around the unit circle starting at 1 (angle 0) and going counter-clockwise. For $n = 4$: $(\omega^0 = 1)$ (0°), $(\omega^1 = i)$ (90°), $(\omega^2 = -1)$ (180°), $(\omega^3 = -i)$ (270°). Mark these four points (right, top, left, bottom of the circle).

Proof pattern — product of all n -th roots of unity (Task 5c, MEMORIZE the technique):

Claim: $\prod_{k=0}^{n-1} \omega^k = +1$ if n is odd, -1 if n is even.

Proof.

$$\prod_{k=0}^{n-1} \omega^k = \omega^{0+1+2+\dots+(n-1)} = \omega^{\frac{n(n-1)}{2}}$$

using the law of exponents $x^a \cdot x^b = x^{a+b}$. Since ω is a primitive n -th root of unity, $\omega^n = 1$.

Case 1: n odd. Then $n-1$ is even, so $(n-1)/2$ is an integer, and

$$\omega^{\frac{n(n-1)}{2}} = (\omega^n)^{\frac{n-1}{2}} = 1^{\frac{n-1}{2}} = 1.$$

****Case 2: n even.**** Write $n = 2m$. Then $n(n-1)/2 = m(n-1) = mn - m$, so

$$\omega^{mn-m} = (\omega^n)^m \cdot \omega^{-m} = 1^m \cdot \omega^{-m} = \omega^{-n/2}.$$

Now $(\omega^{n/2})^2 = \omega^n = 1$, so $\omega^{n/2}$ is a square root of 1. Since ω has order exactly n (it's primitive), $\omega^{n/2} \neq 1$, so it must equal -1 . Hence $\omega^{-n/2} = 1/(-1) = -1$.

Therefore the product is 1 if n odd and -1 if n even. ■

(This proof pattern — "split into odd/even case, use $x^{ab} = (x^a)^b$, use $\omega^n = 1$, identify the unique order-2 element as -1 " — reappears in any roots-of-unity proof. Memorize the structure, not just this instance.)

4.6 Huffman Algorithm

Core intuition: Build an optimal prefix-free binary code by **greedily merging the two least frequent items** at each step, so rare symbols end up deep in the tree (long codes) and frequent symbols end up shallow (short codes).

Algorithm:

1. Put all n symbols in a min-priority queue keyed by frequency.
2. Repeat $n-1$ times: remove the two smallest-frequency nodes (x, y) ; create a new internal node z with $\text{freq}(z) = \text{freq}(x) + \text{freq}(y)$; set $\text{pred}[x] = z, \text{pred}[y] = z$; conventionally the smaller of (x, y) gets $\text{mark} = 0$ (left child), the other gets $\text{mark} = 1$ (right child); insert z back into the queue.
3. The last remaining node is the **root** (no pred).

Worked example (from Task 6 — fully solved):

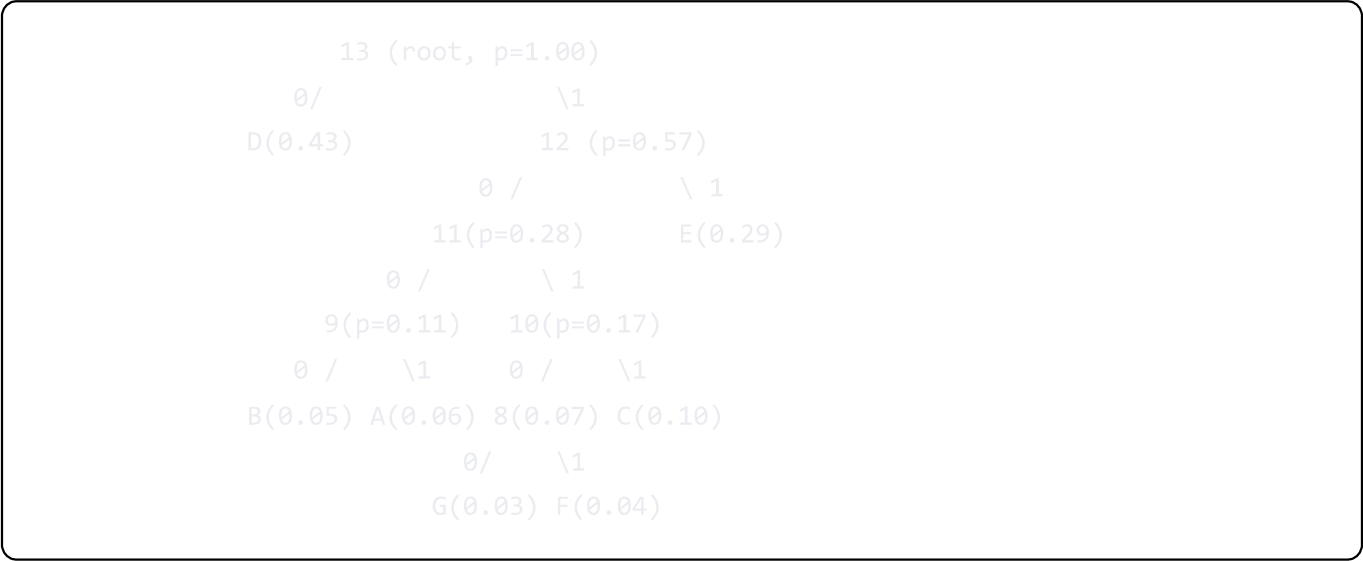
Sorted frequencies: G=0.03, F=0.04, B=0.05, A=0.06, C=0.10, E=0.29, D=0.43 (indices $i=1..7$).

Step	Merge	New node	New weight
1	G(0.03) + F(0.04)	8	0.07
2	B(0.05) + A(0.06)	9	0.11
3	node8(0.07) + C(0.10)	10	0.17
4	node9(0.11) + node10(0.17)	11	0.28
5	node11(0.28) + E(0.29)	12	0.57
6	D(0.43) + node12(0.57)	13 (root)	1.00

Completed table:

i	1 (G)	2 (F)	3 (B)	4 (A)	5 (C)	6 (E)	7 (D)	8	9	10	11	12	13
p_i	0.03	0.04	0.05	0.06	0.10	0.29	0.43	0.07	0.11	0.17	0.28	0.57	1.00
pred	8	8	9	9	10	12	13	10	11	11	12	13	—
mark	0	1	0	1	1	1	0	0	0	1	0	1	—

Resulting coding tree (0 = left, 1 = right):



Cost of a Huffman code:

$$\text{Cost}(T,p) = \sum_{\text{leaves } i} p_i \cdot \text{depth}(i) = \sum_{\text{internal nodes}} (\text{combined weight at merge})$$

$$\text{Cost} = 0.07 + 0.11 + 0.17 + 0.28 + 0.57 + 1.00 = \boxed{2.20}$$

(Sanity check via depths: D at depth1($0.43 \cdot 1 = 0.43$), E depth2($0.29 \cdot 2 = 0.58$), B, A, C depth4($(0.05 + 0.06 + 0.10) \cdot 4 = 0.84$), G, F depth5($(0.03 + 0.04) \cdot 5 = 0.35$). Sum = $0.43 + 0.58 + 0.84 + 0.35 = 2.20 \checkmark$.)

Complexity: $O(n \log n)$ using a binary min-heap priority queue ($(n-1)$ extract-min + insert operations, each $O(\log n)$).

4.7 Minimum Spanning Trees — Kruskal's Algorithm

Definitions (MEMORIZE):

- A **spanning tree** of $G=(V, E, c)$ is an edge set $T \subseteq E$ such that (V, T) is a tree — i.e. **connected and acyclic**, spanning all of V ($|T| = |V| - 1$).
- **Cost of a spanning tree:** $\text{Cost}(T) := \sum_{e \in T} c(e)$.

Kruskal's Algorithm:

1. Sort all edges by weight, ascending.
2. Initialize $R = \emptyset$, and a Union-Find structure with each vertex in its own set.
3. For each edge $e = \{u, v\}$ in sorted order: if $\text{Find}(u) \neq \text{Find}(v)$ (adding e doesn't create a cycle), add e to R and $\text{Union}(u, v)$.
4. Stop when $|R| = |V| - 1$.

The Cut Property (MEMORIZE — this is the correctness theorem behind Kruskal's/Prim's):

Definition. A set $R \subseteq E$ is *extendible* if there exists an MST T of G such that $R \subseteq T$.

Cut Property. Assume the following three conditions hold:

1. $R \subseteq E$ is **extendible**, and
2. $(S, V-S)$ is a cut such that **no edge of R crosses the cut** (every edge of R has both endpoints in S or both in $V-S$), and
3. $e = \{v, w\}$ is an edge of **minimum cost** among all edges crossing the cut, with $v \in S$ and $w \in V-S$.

Then, $R \cup \{e\}$ is also extendible.

Intuition: if you never cross a "boundary" you've committed to avoiding, and you always add the cheapest edge that crosses some safe boundary, you can never be forced away from an MST — this is exactly why Kruskal's greedy edge-by-edge choice is safe.

Executing Kruskal's + Union-Find by hand:

1. Track which edges are already in $\mathbb{R}$ (usually shown in bold) and the current Union-Find forest.
2. The **next edge** is the smallest-weight edge in the sorted order whose two endpoints are in **different** Union-Find components (i.e., adding it does not close a cycle).
3. After adding it, **union the two components**: make the root of one component point to the root of the other (pick a consistent tie-break rule, e.g. smaller root becomes parent, unless your course specifies union-by-rank/size).

Illustrative worked mini-example (method demonstration): suppose components are $\{A, E, F\}$ (root A) and $\{B\}$, $\{C, G\}$ (root C), $\{D, H\}$ (root D), and the next-cheapest edge not yet processed that crosses two different components is $\{F, B\}$ with weight 5. Since F and B are in different components, this edge is valid — add it to $\mathbb{R}$, then update Union-Find so that B 's root now points to A (or vice-versa, per your convention), merging $\{A, E, F\}$ and $\{B\}$ into one component $\{A, B, E, F\}$.

(Apply this exact selection rule — "cheapest edge connecting two different Union-Find components" — to your specific exam graph and weights.)

Complexity: $O(m \log m)$ (or $O(m \log n)$) dominated by sorting the edges; Union-Find operations are near- $O(1)$ amortized (with path compression + union by rank).

4.8 Dijkstra's Algorithm

Core intuition: Maintain tentative shortest-distance estimates $\text{dist}[v]$ from source s . Repeatedly "process" (finalize) the unprocessed vertex with the smallest tentative distance, and use it to **relax** (potentially shorten) the distances of its neighbors.

Runtime with a binary heap priority queue:

$$O((n + m) \log n)$$

n extract-min operations + up to m decrease-key/insert operations, each $O(\log n)$ with a binary heap.)

Execution procedure (fill-in-the-table style): For each "round" i :

1. **Process** vertex x = the unprocessed vertex with smallest $\text{dist}[x]$ (remove it from the priority queue). Record the round number in x 's "processed" field.
2. For every neighbor y of x with edge weight $c(x,y)$: if $\text{dist}[x] + c(x,y) < \text{dist}[y]$, **update** $\text{dist}[y] \leftarrow \text{dist}[x] + c(x,y)$, $\text{pred}[y] \leftarrow x$, and record the round number in which y 's value changed.
3. Continue until all vertices are processed (or the queue is empty).

How to fill the table: for each vertex track three rows — round (when it was last updated / processed), dist (current best known distance from s), pred (the predecessor achieving that distance). Initialize $\text{dist}[s]=0$, all others $\text{dist}=\infty$, $\text{pred} = -$.

Key correctness insight (Task 8c pattern): If, at the end of the algorithm, $\text{dist}[t] = \infty$, this means t is not reachable from s — there is no directed path from s to t in G . (Dijkstra correctly finalizes every reachable vertex to its true shortest distance; unreachable vertices simply never get relaxed away from ∞ .)

Complexity summary table (memorize):

Priority Queue	Runtime
Array (linear scan)	$O(n^2)$
Binary heap	$O((n+m) \log n)$
Fibonacci heap	$O(m + n \log n)$

4.9 TSP Dynamic Programming (Held-Karp) & NP-Completeness

Held-Karp DP meaning of $P[S,w]$ (Task 9a — MEMORIZE):

$P[S,w]$ = the minimum cost of a path that starts at v_1 , visits every vertex in S exactly once (in some order), and ends at vertex w (where $w \notin S$, $S \subseteq V \setminus \{v_1\}$).

Base case: $P[\emptyset, w] = c(v_1, w)$. Recurrence: $P[S,w] = \min_{u \in S} P[S \setminus \{u\}, u] + c(u, w)$.

Final answer: $\min_w P[V \setminus \{v_1, w\}, w] + c(w, v_1)$.

Complexity: $O(n^2 \cdot 2^n)$ — exponential, but far better than brute-force $O(n!)$.

NP membership proofs — MEMORIZE this exact template:

To show a decision problem Π is in NP, you must exhibit:

1. A **certificate** (candidate solution) of size **polynomial** in the input size.

2. A **polynomial-time verification algorithm** that, given the input and the certificate, checks whether the certificate is valid (i.e., correctly certifies a "yes" instance).

Worked proof — Vertex Cover Decision Problem $\in$ NP:

Certificate: a set $C \subseteq V$ with $|C| \leq k$. **Verification:** (1) check $|C| \leq k$ — $O(n)$. (2) for every edge $\{u, v\} \in E$, check that $u \in C$ or $v \in C$ — $O(m)$ using a boolean membership array for C . Both checks run in time polynomial in $|V| + |E|$, and the certificate itself has size $O(n)$ (polynomial). Hence Vertex Cover Decision $\in$ NP. ■

Worked proof — Minimal Spanning Tree Decision Problem $\in$ NP:

Certificate: an edge set $T \subseteq E$. **Verification:** (1) check $|T| = |V| - 1$ — $O(1)$. (2) check (V, T) is connected and acyclic, e.g. via Union-Find or BFS/DFS — $O(n+m)$. (3) compute $\text{Cost}(T) = \sum_{e \in T} c(e)$ and check $\text{Cost}(T) \leq k$ — $O(n)$. All steps are polynomial, and the certificate size is $O(n)$. Hence MST Decision $\in$ NP. ■ (Note: MST-Decision is actually solvable in polynomial time — i.e. it's in P — but $P \subseteq NP$, so showing NP membership via a certificate is still valid and is what's being asked.)

General framework for any "prove $X \in NP$ " question:

1. State the certificate clearly (what does a "yes" witness look like?).
2. State its size is polynomial in the input.
3. List each verification check and its runtime.
4. Conclude: total verification time is polynomial $\Rightarrow X \in NP$.

5. SOLVED EXAM QUESTIONS — QUICK REFERENCE ANSWER KEY

Task Answer

1a Divide, Conquer, Combine (with descriptions above)

1b $T[n] \leq T[\lfloor n/2 \rfloor] + T[\lfloor n/2 \rfloor] + O(n)$

1c See QuickSort pseudocode §4.1

2a Level k : 5^k calls, size $(n/3^k)$, merge cost $(c \cdot \log(n/3^k))$; depth $(d = \log_3 n)$

2b $T[n] = \sum_{k=0}^{\log_3 n - 1} 5^k \cdot c \cdot \log(n/3^k) + n^{\log_3 5} \cdot c$

3a-d See §4.3

4a $\text{MAXIS}(v)$ = size of max independent set in subtree T_v

Task Answer

-
- 4b Answer = $\text{MAXIS}(r)$
-
- 4c $\text{MAXIS}(v) = \max(1 + \sum \text{MAXIS}(\text{grandchildren}), \sum \text{MAXIS}(\text{children}))$
-
- 4d Method in §4.4 — apply bottom-up to your tree
-
- 5a Evaluate (FFT) → Pointwise multiply → Interpolate (Inverse FFT), all $O(n \log n)$ / $O(n)$
-
- 5b $n = 4$; roots at $1, i, -1, -i$
-
- 5c Proof in §4.5
-
- 6a-c Table, tree, Cost = **2.20** (see §4.6)
-
- 7a Spanning tree = $T \subseteq E$ s.t. (V, T) connected+acyclic; $\text{Cost}(T) = \sum c(e)$
-
- 7b Cut Property statement in §4.7
-
- 7c Method in §4.7 — apply to your graph
-
- 8a $O((n+m) \log n)$
-
- 8b Method in §4.8 — apply to your graph
-
- 8c t unreachable from s (no path exists)
-
- 9a $P[S, w]$ = min cost path from v_1 through all of S ending at w
-
- 9b NP proofs in §4.9
-

6. THEORY TO MEMORIZE (flashcard list)

- **3 phases of D&C:** Divide, Conquer, Combine.
- **MergeSort recurrence:** $T[n] \leq T[\lfloor n/2 \rfloor] + T[\lfloor n/2 \rfloor] + O(n)$.
- **Recursion tree formula:** level $k \rightarrow a^k$ calls of size n/b^k ; depth $d = \log_b n$; leaves $= n^{\{\log_b a\}}$.
- **Edit distance def:** min insertions+deletions+substitutions to transform A into B.
- **MCM def:** minimize scalar multiplications over parenthesizations.
- **TSP def:** min-cost Hamiltonian cycle.
- **Independent Set def:** no two chosen vertices share an edge.
- **MAXIS(v) recursion:** $\max(1 + \sum \text{grandchildren}, \sum \text{children})$.

- **FFT 3 phases:** Evaluate $\rightarrow$ Pointwise multiply $\rightarrow$ Interpolate; total $O(n \log n)$.
 - **n -th roots of unity product:** $+1$ if n odd, -1 if n even.
 - **Huffman:** always merge two smallest; cost = sum of all internal node weights.
 - **Spanning tree def:** $T \subseteq E$, (V, T) connected and acyclic.
 - **Cut Property:** extendible R + cut respecting R + min-cost crossing edge $\Rightarrow R \cup \{e\}$ extendible.
 - **Dijkstra runtime (binary heap):** $O((n+m) \log n)$.
 - $\text{dist}[t] = \infty$ at termination $\Rightarrow t$ unreachable from s .
 - $P[S, w]$ (TSP-DP): min cost path $v_1 \rightarrow (\text{all of } S) \rightarrow w$.
 - **NP proof template:** certificate (poly size) + poly-time verifier $\Rightarrow \text{problem} \in \text{NP}$.
-

7. PROBLEM-SOLVING FRAMEWORKS

A) "Name/describe the phases of algorithm X" $\rightarrow$ Always structure as Divide/Conquer-style breakdown or, for FFT, Evaluate/Multiply/Interpolate. State *what happens* and *its runtime* for each phase.

B) "Build the recursion tree for $T[n] = aT[n/b] + f(n)$ " $\rightarrow$

1. Write the generic row: level $k \rightarrow a^k$ calls, size n/b^k .
2. Multiply by per-call merge cost $f(n/b^k)$ to get level-total cost.
3. Solve $n/b^d = 1$ for depth d .
4. Sum levels 0 to $d-1$, add leaf-level cost $a^d \cdot T[1]$.

C) "Execute a greedy/graph algorithm on a small instance" $\rightarrow$

1. Write out the algorithm's invariant/rule explicitly (e.g., "always merge two smallest" or "always pick cheapest edge crossing components" or "always process smallest tentative distance").
2. Go step by step, updating *all* auxiliary structures (Union-Find, priority queue, pred/dist tables) at every single step — partial credit is given per correct intermediate state, not just the final answer.
3. Double check termination condition ($|R| = n-1$ for MST; queue empty for Dijkstra; all merged into 1 node for Huffman).

D) "Formalize a DP problem / recursive formula" $\rightarrow$

1. Define the sub-problem in words first ($\text{MAXIS}(v) = \dots$, $P[S, w] = \dots$).
2. State what a valid solution "commits to" (include vs exclude, path structure, etc.).

3. Write the recurrence as a $\boxed{\max}/\boxed{\min}$ over the finitely many valid choices.
4. State the base case and how the final global answer is read off (e.g., root value, or min/max over all $\boxed{w}$).

E) "Prove a problem is in NP" →

1. Certificate — what does the witness look like?
2. Certificate size is polynomial — say why.
3. Verifier steps + each one's runtime.
4. Conclude total = polynomial $\Rightarrow \in \text{NP}$.

F) "Prove an algebraic identity about roots of unity / complex numbers" →

1. Rewrite the product/sum using exponent laws.
 2. Split into cases if parity (odd/even) matters.
 3. Use $\boxed{\omega^n = 1}$ and the fact that $\boxed{\omega}$ has order exactly $\boxed{n}$ to identify special values (e.g., $\boxed{\omega^{\{n/2\}} = -1}$).
-

8. PRACTICE PLAN (time-boxed)

If you have ~6-8 hours before the exam:

Time block	Focus	Activity
1 hr	Task 5 (FFT)	Rewrite the 3 phases from memory; redo the roots-of-unity proof twice unaided
1 hr	Task 4 (Tree DP)	Draw 2–3 small trees yourself, execute TreeMaxIS bottom-up from scratch
45 min	Task 6 (Huffman)	Build 2 fresh frequency tables (5–7 symbols), run the algorithm fully by hand
1 hr	Task 7 (MST/Kruskal)	Memorize Cut Property word-for-word; run Kruskal's + Union-Find on 2 small graphs
45 min	Task 3 (Definitions)	Flashcard all 6 Input/Output definitions until instant recall
45 min	Task 9 (NP + TSP-DP)	Rewrite both NP proofs from memory; recite $P[S, w]$ definition
45 min	Task 8 (Dijkstra)	Run Dijkstra by hand on 2 small digraphs, fill dist/pred/round tables
30 min	Task 1 & 2 (D&C, recursion trees)	Rewrite QuickSort pseudocode from memory; redo one recursion-tree derivation
30 min	Full mixed review	Skim §6 flashcard list top to bottom, out loud

If you have <2 hours: Do only §6 (Theory to Memorize) cold, then the FFT proof, the Huffman worked example, and the Cut Property statement — these cover ~40% of total marks with the least time investment.

9. CHEAT SHEET (last-minute glance before exam)

D&C: Divide → Conquer → Combine
 MergeSort: $T[n] \leq T[\lceil n/2 \rceil] + T[\lfloor n/2 \rfloor] + O(n)$
 Recursion tree: level $k \rightarrow a^k$ calls, size n/b^k ; depth $d = \log_b(n)$; leaves $= n^{(\log_b a)}$
 Edit Distance: min ins/del/sub to turn A into B
 MCM: min scalar mults over parenthesization
 TSP: min-cost Hamiltonian cycle

Indep. Set: $S \subseteq V$, no two vertices adjacent

Tree MAXIS(v): $\max(1 + \sum \text{MAXIS}(\text{grandchildren}), \sum \text{MAXIS}(\text{children}))$
 $\text{answer} = \text{MAXIS}(\text{root})$

FFT mult: Evaluate(FFT) $\rightarrow$ Pointwise multiply $\rightarrow$ Interpolate(IFFT), $O(n \log n)$
 $n = \text{next power of } 2 \geq \deg(A) + \deg(B) + 1$
 $\prod \omega^k \text{ over } k=0..n-1 = +1 \text{ (n odd)} / -1 \text{ (n even)}$

Huffman: repeatedly merge 2 smallest; Cost = $\sum$ (all internal node weights)

Spanning tree: $T \subseteq E$, (V, T) connected+acyclic; Cost(T) = $\sum c(e)$

Cut Property: R extendible + cut respects $R + e = \text{min-cost crossing edge}$
 $\Rightarrow R \cup \{e\}$ extendible

Kruskal's: sort edges; add if endpoints in different Union-Find sets

Dijkstra: $O((n+m) \log n)$ w/ binary heap
 $\text{dist}[t] = \infty$ at end $\Rightarrow t$ unreachable from s

$P[S, w]$ (TSP-DP): min cost of path $v_1 \rightarrow \text{all of } S \rightarrow w$

NP proof: certificate (poly size) + poly-time verifier $\Rightarrow \in \text{NP}$

10. FINAL EXAM STRATEGY

1. **Grab the free points first.** Task 3 (definitions), Task 1a (D&C phases), Task 7a (spanning tree def), Task 8a (Dijkstra runtime) are pure recall — do these in the first 5 minutes while your memory is freshest.
2. **Show your work on every execution question** (Kruskal, Dijkstra, Huffman, TreeMaxIS). Partial credit is awarded per correct intermediate table/tree state, not just the final number — never erase your working steps.
3. **On recurrence/recursion-tree questions**, always write the general row (level $k \rightarrow a^k$ calls of size n/b^k) even before plugging in numbers — many rubrics award marks for the correct general structure independent of arithmetic slips.
4. **For proofs (roots of unity, NP membership)**, state the theorem/fact you're invoking explicitly (e.g., "since ω is a primitive n -th root of unity, $\omega^n = 1$ ") — examiners give marks for correctly *citing* the tool, not just using it silently.
5. **Watch your time budget** proportionally to marks: spend roughly $(\text{task marks} / 90) \times \text{total exam time}$ on each task. Don't let a single hard proof (e.g., Task 5c, 7 pts) eat time you need for higher-value execution tasks.

6. **Double check base cases** on every DP/recursion question ($T[1]=c$), leaf ($MAXIS=1$), ($P[\emptyset, w]=c(v1, w)$) — these are easy to forget under pressure and are often explicitly graded.
7. For "fill in the blank" theorem statements (like the Cut Property), if you blank on exact wording, write the *idea* in your own words and label it clearly — partial credit for conceptual correctness is common even without verbatim phrasing.